

VERIFIER-GROUNDED HIERARCHICAL SKILL EVOLUTION FOR SOFTWARE ENGINEERING AGENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Software-engineering agents leave behind rich traces: issue interpretations, file searches, edits, tests, failures, and recoveries. Turning these traces into reusable knowledge is attractive, but raw trace retrieval risks task leakage and one-shot summaries can introduce harmful or over-specific instructions. We propose HSE, a verifier-grounded framework that evolves external textual skills for frozen coding agents. HSE separates single-task evidence from reusable skills, promotes evidence through a hierarchy of repo-level and cross-repo failure-mode skills, and trains separate Skill Writer and Skill Evaluator policies under a fixed harness policy. During SWEGym training, the official verifier is used only for outcome supervision, evaluator calibration, and validation gates. During downstream evaluation, verifier labels are not available for updating memory. We describe a strict no-exact-transfer protocol and an online test-time evolution protocol over a 2×2 executor matrix: GLM-5.2 and GPT-5.5, each run in Pi and Claude-style agent harnesses. This matrix lets us test model transfer, harness transfer, and joint model+harness transfer. Final resolved rates and ablations are intentionally left blank in this draft pending the full experimental run.

1 INTRODUCTION

Large language model agents can now repair real software bugs by reading a repository, editing code, and running tests. Yet each successful or failed attempt produces information that is usually discarded after the benchmark trial. A natural next step is to convert agent trajectories into external skills: compact textual procedures that tell future agents how to localize, edit, validate, or recover on similar software tasks.

This conversion is not straightforward. A raw trajectory can contain task-specific paths, accidental patch shapes, transient environment errors, and implicit information about the benchmark instance. If such details are injected into a future prompt, the system may improve by memorization rather than by transfer. Conversely, a generic summary such as “run focused tests” is often too weak to change behavior. The central question of this paper is therefore: *how can a coding-agent system learn reusable software-engineering skills from training-time verifier feedback without using test-time verifier feedback or leaking task-specific solutions?*

We propose HSE: **hierarchical skill evolution**. The key design choice is to not call every trace artifact a skill. A task attempt first becomes *task evidence*: a structured record of public signals such as touched paths, edited files, tests, diagnostics, selected skills, failure signatures, and verifier outcomes. A Skill Writer policy then drafts candidate repo-level skills only from multiple task evidence records in the same repository. A candidate failure-mode skill is drafted only from repeated patterns across repositories, such as localization drift, weak validation, no-diff exits, or timeout recovery. A Skill Evaluator policy scores each candidate for utility, support, and risk, and decides whether to accept, reject, revise, or keep it as memory-only evidence.

The framework is built around a hard information boundary. SWEGym verifiers are available during training, so we use them to label outcomes, calibrate the evaluator, and gate candidate skill states on held-out validation tasks. On SWE-Bench Verified and other downstream benchmarks, no official verifier label, hidden test result, gold patch, or future task outcome may influence skill

updates. This makes the resulting protocol suitable for testing whether SWEGym-trained skills and writer/evaluator policies transfer to new software tasks.

Our contributions are:

- We formulate software-agent skill evolution as a hierarchical evidence-gated process that separates task evidence, repo-level skills, cross-repo failure-mode skills, and writer/evaluator policies.
- We introduce a verifier-boundary protocol: training may use SWEGym verifier outcomes, while downstream test-time evolution is gated only by a frozen evaluator over public trace signals.
- We give an implementation for the Pi coding-agent harness with an OpenAI-compatible model interface, Harbor/E2B execution, versioned skill archives, promotion logs, validation gates, and deterministic evaluator shells that can be replaced by learned policies.
- We specify the ICLR experimental protocol for GLM-5.2 and GPT-5.5 in both Pi and Claude-style harnesses, including model-transfer and harness-transfer tests, with exact-task memory reported only as an upper-bound adaptation setting.

2 PROBLEM SETTING

Tasks and agents. Let x be a software-engineering task consisting of a natural-language issue, a repository snapshot, and a public execution environment. A frozen coding agent M interacts with a harness H by issuing file reads, searches, edits, shell commands, and final answers. A trajectory τ contains the observable history: prompts, tool calls, outputs, diffs, public tests, and runtime diagnostics.

Training-time verifier. During SWEGym training we have a verifier V that maps a completed trial to a reward $r \in \{0, 1\}$ or a diagnostic outcome. We use V as supervision for training-time labels and validation gates, but not as a writer of skills and not as a provider of hidden solutions. At downstream test time, V is unavailable for memory updates. It is used only after the protocol completes to report final benchmark scores.

Skill state. The external state S contains structured evidence and accepted skills. We use four deployed categories:

- **Task evidence** records what happened on one task. It is not a reusable skill.
- **Repo skills** summarize repeated, evidence-gated procedures within a repository.
- **Failure-mode skills** summarize cross-repo recovery strategies for repeated agent failures.
- **General seed skills** provide weak process priors for common stages such as reproduce, localize, edit, validate, and recover.

The writer policy π_w drafts evidence summaries and candidate skills. The evaluator policy π_e assigns decisions, proxy rewards, confidence, and risk flags.

3 HIERARCHICAL SKILL EVOLUTION

3.1 STATE HIERARCHY

HSE uses different update clocks for different abstraction levels. Lower levels can change quickly because they affect only local evidence. Higher levels change slowly because they can influence future tasks.

Task evidence stores public fields such as task id, repo, problem summary, reward, previous reward, case label, exception type, selected skills, touched paths, edited paths, public test commands, failure signature, and diagnostic signals. It may feed higher-level aggregation, but it is not rendered as a reusable `SKILL.md` artifact by default.

Table 1: Skill-evolution objects and update clocks. Main experiments freeze all training-derived transferable state before downstream evaluation.

Level	Object	Update clock	Evidence requirement
L0	Rollout trace	Every task attempt	Public trajectory and runtime outputs
L1	Task evidence	Every task attempt	One task trace; verifier label during training only
L2	Repo skill candidate	Same-repo batch	Multiple task evidence records from one repo
L3	Failure-mode skill candidate	Cross-repo trigger	Repeated repo-level summaries across repos
L4	Skill Writer policy	Slow policy window	Aggregated accepted/rejected histories
L5	Skill Evaluator policy	Slow policy window	Verifier-calibrated false accepts/rejects

Agent trace $\rightarrow$ **Task evidence** $\rightarrow$ **Repo candidate** $\rightarrow$ **Failure-mode skill**. The trace contains tool calls, edits, tests, and diagnostics. Task evidence is a public structured record, not a deployed transferable skill. Repo candidates summarize same-repo repeated paths, tests, and failure signals. Failure-mode skills summarize cross-repo recovery rules with triggers and stop conditions. **Evaluator gate**: each promotion step must pass support thresholds, risk checks, and validation constraints before it becomes active in the SkillRepo.

Figure 1: **AI-friendly caption**: A left-to-right pipeline showing how a software-engineering agent trace is converted into task evidence, then into a repo-level candidate, then into a cross-repo failure-mode skill. The visual should emphasize that task evidence is not directly deployed as a transferable skill; every upward arrow is guarded by an evaluator gate that checks support, risk, and validation.

3.2 WRITER AND EVALUATOR

The writer policy produces structured candidates. For a repo candidate, the writer receives a cluster of same-repo task evidence and emits the fields name, trigger, evidence gate, actions, validation hint, abort condition, and support summary. For a failure-mode candidate, it receives a cross-repo cluster and emits a process-level recovery skill with a failure signature, trigger, actions, stop condition, and support summary.

$$c_{\text{repo}} = f_w(E_{\text{repo}}), \quad c_{\text{fail}} = f_w(E_{\text{cross-repo}}).$$

Both candidate types are deliberately structured so the evaluator can inspect their triggers, evidence support, actions, and risks before deployment.

The evaluator implements the boundary between candidate generation and deployed skills:

$$\pi_e(c, E) \rightarrow (d, \hat{r}, \rho, z),$$

where $d \in \{\text{ACCEPT}, \text{REJECT}, \text{REVISE}, \text{MEMORY-ONLY}\}$, $\hat{r}$ is a proxy reward, ρ is confidence, and z is a set of risk flags. Hard safety risks include hidden verifier details, oracle patches, exact patches, task-id memorization, and private sandbox paths. Repo candidates require repeated same-repo support; failure-mode candidates require support from multiple distinct repositories.

Our current implementation uses a deterministic evaluator shell with explicit thresholds and risk checks. This is a deliberate interface choice: it makes the contract auditable now, while allowing the same function call to be replaced by a learned evaluator policy trained on SWEgym calibration events.

Training on SWEgym:

rollout $\rightarrow$ task evidence $\rightarrow$ repo/failure candidates $\rightarrow \pi_e$ gate $\rightarrow$ validation gate.

The SWEgym verifier labels outcomes and validates candidate skill states.

Downstream evaluation:

frozen (π_w, π_e , general/failure skills) $\rightarrow$ agent rollout $\rightarrow$ public evidence only.

No official downstream verifier label, hidden test, gold patch, or future task outcome is used for updating skills.

Figure 2: **AI-friendly caption:** A two-lane diagram contrasting training and downstream evaluation. The top lane is SWEgym training, where verifier labels supervise case labels, evaluator calibration, and validation gates. The bottom lane is downstream testing, where the writer, evaluator, and transferable skills are frozen and only public trace signals can be used. The verifier icon must appear only in the training lane and final reporting, not in the update path for downstream tests.

3.3 FIXED HARNESS POLICY

The learned writer and evaluator are not allowed to invent their own reward labels, update clocks, or promotion budgets. A fixed harness policy maps verifier transitions and runtime diagnostics to case labels:

- **Strong positive:** previous attempt failed and current attempt passed.
- **Weak positive:** current attempt passed without a paired improvement, or a previously passing attempt remains passing.
- **Strong negative:** previous attempt passed and current attempt failed.
- **Weak negative:** current attempt failed without a paired regression.
- **Diagnostic:** missing reward, timeout, setup error, verifier error, sandbox failure, or provider/runtime failure.

Cold-start solved traces are weak positives; unsolved traces are weak negatives unless a diagnostic is present. Strong positives can increase support for a candidate, but cannot directly create a global skill. Strong negatives prioritize rollback, trigger narrowing, and evaluator false-accept penalties. Diagnostics are kept out of semantic edit skills and are used only for runtime or recover/validate safeguards.

3.4 TRAINING LOOP

The training loop alternates between rollouts, evidence extraction, candidate generation, evaluator gating, and validation.

Default update hyperparameters. Unless otherwise stated, repo updates use same-repo batches of 5 task evidence events, failure-mode updates trigger every 4 repo updates, failure-mode promotion requires at least 3 supporting repositories, and policy updates are limited to at most 3 bullet edits per policy window. Each repo batch emits at most one repo candidate by default; each failure-mode trigger promotes at most two accepted failure-mode skills. These values match the current harness defaults and are treated as fixed protocol hyperparameters.

3.5 VALIDATION GATE

Candidate skill states are evaluated on a repo-isolated SWEgym validation set. We compute:

$$\text{effective_resolved_rate} = \frac{\text{resolved valid trials}}{\text{effective valid trials}}, \quad \text{diagnostic_rate} = \frac{\text{diagnostic trials}}{\text{total trials}}.$$

A candidate state is accepted only if it has enough effective validation trials, does not exceed the diagnostic-rate threshold, and does not regress relative to the current accepted state beyond a fixed tolerance. Otherwise, the candidate version is rejected or rolled back, and the decision is logged.

4 EXPERIMENTS

This section specifies the experiment settings. Numeric results are reserved until the full ICLR run finishes.

4.1 DATASETS

SWEGym training and validation. We use a 500-task SWEGym subset as the evolution source. The split is repo-isolated: repositories selected for validation are held out from the training split, with the validation size chosen closest to 5% of the subset. In the planned main run this yields 474 training tasks and 26 validation tasks. The SWEGym verifier is available only in this phase.

SWE-Bench Verified final evaluation. The primary downstream benchmark is SWE-Bench Verified. No exact SWE-Bench task trajectory, official verifier label, hidden test output, or gold patch is used to update skills during the evaluated run. The official verifier is invoked only after the run to report resolved count and resolved rate.

Optional out-of-distribution evaluation. We will also report transfer on SWE-Bench Pro and Terminal-Bench 2.0 when the runs are complete. Terminal-Bench uses the same Harbor/E2B execution wrapper but a terminal-task prompt.

4.2 MODELS AND HARNESSES

Executor matrix. We evaluate two executor models, GLM-5.2 (glm52) and GPT-5.5, each under two agent harnesses: Pi Coding Agent and a Claude-style coding-agent harness. All four cells use the same Harbor/E2B sandbox interface, benchmark task selection, task ordering, timeout budget, validation gate, and no-test-verifier update rule. Where model APIs differ, we use the same OpenAI-compatible provider abstraction and record the endpoint type in the run config.

Model and harness transfer. The 2×2 matrix lets us distinguish three transfer questions. First, **model transfer**: skills and writer/evaluator policies trained with one model are evaluated with the other model under the same harness (GLM-5.2→GPT-5.5 and GPT-5.5→GLM-5.2). Second, **harness transfer**: skills trained under one harness are evaluated under the other harness with the same model (Pi→Claude-style and Claude-style→Pi). Third, **joint transfer**: both the model and harness change between evolution and evaluation. We report these separately from within-cell evolution, where train and test use the same model/harness cell.

Tools and controls. The agent tool surface is fixed within each harness family and exposes the same software-engineering operations: read, write, edit, bash, grep, find, and ls. Prompts may differ between Pi and Claude-style harnesses, but task order, sandbox resources, retry policy, and acceptance rules are held fixed. Each run logs the active skill version, source model/harness cell, target model/harness cell, selected skills, runtime diagnostics, verifier reports, and score reports.

Execution controls. All compared settings use the same dataset version, task order, sandbox resources, agent timeout, retry policy, and concurrency within a model/harness pair. We log job directories, configs, active skill versions, selected skills, tool traces, runtime diagnostics, verifier reports, and score reports.

4.3 PROTOCOLS

4.4 METRICS

The primary metric is resolved rate. We also report resolved-count deltas, paired transition counts (0→1, 1→0, 1→1, 0→0), diagnostic rate, no-diff rate, timeout rate, weak-validation rate, localization-drift rate, selected-skill read rate, trigger-match rate, retrieval false-positive rate, tool-call count, token count, wall-clock time, and validation-command count. These metrics are required because a skill system can improve pass rate while increasing regressions, latency, or prompt cost.

Table 2: Main evaluation protocols. Exact-task skills are reported only as an adaptation upper bound and are not used to support transfer claims.

Setting	Uses exact task memory?	Updates during test?	Purpose
No Skills	No	No	Frozen-agent baseline
Public SWE Skills	No	No	Static public skill baseline
Raw Trace Retrieval	No	No	Tests whether raw trajectory memory is enough
One-shot StageSkill	No	No	Tests ungated trace-to-skill summarization
Frozen Failure Skills	No	No	Tests SWEGym-trained transferable failure-mode skills
Frozen Writer/Evaluator + Online Repo Skills	No	Yes, public evidence only	Tests verifier-free online evolution
Same-budget Retry	No	No structured skills	Controls for extra attempts or compute
Exact Task Skills	Yes	Optional	Upper bound/adaptation only

4.5 RESULT TABLES TO FILL

Table 3: Primary SWE-Bench Verified within-cell results. Values are intentionally blank until the full GLM-5.2/GPT-5.5 by Pi/Claude-harness matrix finishes.

Model/Harness/Setting	Resolved	Rate	0→1	1→0	Diag.
GLM-5.2 / Pi / No Skills	TBD	TBD	TBD	TBD	TBD
GLM-5.2 / Pi / HSE	TBD	TBD	TBD	TBD	TBD
GLM-5.2 / Claude harness / No Skills	TBD	TBD	TBD	TBD	TBD
GLM-5.2 / Claude harness / HSE	TBD	TBD	TBD	TBD	TBD
GPT-5.5 / Pi / No Skills	TBD	TBD	TBD	TBD	TBD
GPT-5.5 / Pi / HSE	TBD	TBD	TBD	TBD	TBD
GPT-5.5 / Claude harness / No Skills	TBD	TBD	TBD	TBD	TBD
GPT-5.5 / Claude harness / HSE	TBD	TBD	TBD	TBD	TBD

5 RELATED WORK

Software-engineering benchmarks. SWE-Bench introduced real GitHub issue resolution as a benchmark for language models and coding agents (Jimenez et al., 2024). SWE-Bench Verified refines the benchmark with human-verified tasks and evaluation criteria (OpenAI, 2024). SWE-Gym provides executable training environments for software-engineering agents and is the source of verifier-supervised skill evolution in our work (Pan et al., 2024). Terminal-Bench studies broader terminal workflows beyond repository patching (Merrill et al., 2026).

Language-agent memory and reflection. Prior language-agent systems use verbal feedback, self-reflection, or memory to improve future behavior (Yao et al., 2023; Shinn et al., 2023; Wang et al., 2023). HSE differs by enforcing a benchmark-verifier boundary and by separating task-local evidence from transferable repo and failure-mode skills.

Table 4: Transfer matrix to fill after the full run. Source denotes the model/harness cell used for skill evolution; target denotes the cell used for SWE-Bench Verified evaluation.

Source cell	Target cell	Transfer type	Resolved delta
GLM-5.2 / Pi	GPT-5.5 / Pi	Model transfer	TBD
GPT-5.5 / Pi	GLM-5.2 / Pi	Model transfer	TBD
GLM-5.2 / Claude harness	GPT-5.5 / Claude harness	Model transfer	TBD
GPT-5.5 / Claude harness	GLM-5.2 / Claude harness	Model transfer	TBD
GLM-5.2 / Pi	GLM-5.2 / Claude harness	Harness transfer	TBD
GLM-5.2 / Claude harness	GLM-5.2 / Pi	Harness transfer	TBD
GPT-5.5 / Pi	GPT-5.5 / Claude harness	Harness transfer	TBD
GPT-5.5 / Claude harness	GPT-5.5 / Pi	Harness transfer	TBD
GLM-5.2 / Pi	GPT-5.5 / Claude harness	Joint transfer	TBD
GPT-5.5 / Claude harness	GLM-5.2 / Pi	Joint transfer	TBD

Table 5: Ablation plan. Each row removes one mechanism while keeping model, harness, task order, and budget fixed.

Ablation	Question	Resolved delta
Without evaluator gate	Do ungated candidates introduce harmful skills?	TBD
Without validation gate	Does repo-isolated validation prevent regressions?	TBD
Without failure-mode skills	Are cross-repo recovery rules useful?	TBD
Without repo skills	Are same-repo procedures useful in online evolution?	TBD
Without policy update windows	Do slow clocks reduce drift and prompt bloat?	TBD
Raw trace retrieval only	Is structured skill abstraction necessary?	TBD

Skill extraction and text-space optimization. Recent systems distill trajectories into textual skills, optimize external instructions, or maintain skill repositories for agents (Ni et al., 2026; Yang et al., 2026a; Ouyang et al., 2026; Yang et al., 2026b). Our focus is the software-engineering setting, where executable verifiers are available during training but not during test-time memory updates, and where exact task leakage is a central evaluation risk.

6 LIMITATIONS

This draft intentionally leaves experimental results blank. The method also has several substantive limitations. First, verifier-grounded training is only as good as the verifier and validation split; incomplete public tests can still miscalibrate evaluator decisions. Second, failure-mode skills may be too broad if failure signatures are noisy. Third, online test-time evolution is order sensitive, so we must report original-order and random-order robustness. Fourth, the current evaluator implementation is deterministic and rule-based; a learned evaluator must be tested against false accepts, false rejects, and prompt bloat. Finally, the full 2×2 transfer matrix is a planned setting. Within-cell

results, model-transfer results, harness-transfer results, and joint-transfer results must be reported separately because they answer different questions.

7 CONCLUSION

We presented HSE, a verifier-grounded approach for evolving external software-engineering skills for frozen coding agents. The framework promotes public task evidence through repo-level and cross-repo failure-mode gates, separates writer and evaluator policies, and freezes transferable state before downstream evaluation. The resulting protocol is designed to answer whether SWEGym can train reusable skill-writing and skill-selection behavior that transfers to SWE-Bench Verified, SWE-Bench Pro, and Terminal-Bench without test-time verifier feedback.

REFERENCES

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjin Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills, 2026.
- OpenAI. SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024. Human-validated subset of SWE-bench.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. SkillOS: Learning skill curation for self-evolving agents, 2026.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills, 2026a.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. AutoSkill: Experience-driven lifelong learning via skill self-evolution, 2026b.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A IMPLEMENTATION NOTES

The implementation stores accepted skills under versioned directories `skills/accepted/vXXXX/`. Repo skills are written under `_repos/`, failure-mode skills under `_failure_modes/`, and general SWE seeds under `_general/swe/`. Training artifacts include `task_events.jsonl`, `repo_clusters.jsonl`, `failure_clusters.jsonl`, `evaluator_decisions.jsonl`, `evaluator_calibration.jsonl`, `promotion_decisions.jsonl`, `harness_policy.md`, `generator_policy.md`, and `evaluator_policy.md`.

B COMMAND SKELETONS

```
# SWEGym transfer loop, one source cell.
python scripts/run_swegym_skill_evo_loop.py \
  --run-name swegym_<model>_<harness>_hse_main \
  --provider openai \
  --provider-model <model-id> \
  --provider-api openai-completions \
  --concurrency <N>

# SWE-Bench Verified target-cell evaluation with accepted skills.
python scripts/run_skill_evo_verified.py \
  --run-name verified_<source>_to_<target>_hse_final \
  --dataset swe-bench/swe-bench-verified@2 \
  --provider openai \
  --provider-model <target-model-id> \
  --all-verified \
  --concurrency <N>
```